Karbantartási Napló Biztonsági Profil

https://gemini.google.com/app/dd89e891152e2439

User prompt: Szia! Egy saját használatú, 2 fős belső karbantartási napló mikro-alkalmazást készítek Firebase-en (admin által létrehozott fiókokkal, fizetés és külső ügyfél-PII nélkül), ahol a felhasználók saját `maintenanceLogs` bejegyzéseket hozhatnak létre közvetlenül kliensoldalról. Kérlek, a Firebase Web & PWA Security Framework v1.1.0 alapján: 1. Állítsd össze a projekt Kompakt L1 `SECURITY_PROFILE.md` profilját magyarul (`CORE-TIER-L1-001`)!2. Írd meg hozzá a szigorú `firestore.rules` szabályt (`hasOnly`, típus- és méretkorlátok, immutabilitás, `.limit(50)`) és a kötelező kliens lekérdezési mintát (`REF-RULES-BASE-001`)! 3. Add meg a kötelező pozitív és negatív (`assertFails`) emulátoros teszteseteket!

Response: A **Firebase Web & PWA Security Framework (v1.1.0)** előírásai (`CORE-TIER-L1-001`, `REF-RULES-BASE-001`, `REF-TEST-RULES-001`) alapján az alábbiakban található a 2 fős belső karbantartási napló mikro-alkalmazás kompakt biztonsági profilja, a szigorú Firestore Security Rules és kliensoldali lekérdezési minta, valamint az emulátoros egységteszt-csomag.

1. Kompakt L1 `SECURITY_PROFILE.md` (`CORE-TIER-L1-001`)

Az L1 Lightweight Mode (`CORE-TIER-L1-001`) adminisztratív egyszerűsítést biztosít, de **nem jelent biztonsági felmentést**: a tulajdonosi ellenőrzés (`isOwner`), a sémavalidáció (`hasOnly`, típus/méret, módosíthatatlanság), a titokvédelem és a negatív biztonsági tesztek (`assertFails`) kötelezőek maradnak.

```
# SECURITY_PROFILE.md – Belső Karbantartási Napló Mikro-Alkalmazás (v1.1.0)

**Keretrendszer verzió:** `1.1.0` (Utolsó felülvizsgálat: `2026-11-03` | Következő 90 napos audit: `2027-02-01`)
**Projekt:** Belső Karbantartási Napló (`maintenanceLogs`)
**Repository:** `UNKNOWN` (lokális / belső repo)
**Dátum:** `2026-11-12`
**Felelős (Projektgazda):** Projekt Tulajdonos
**Tervező / Kódoló / Ellenőrző AI-modell(ek):** Gemini (`SELF_REVIEWED_L1_ONLY` L1-ben megengedett)

---

## Kompakt L1 Profil Blokkminta (`CORE-TIER-L1-001`)

- **Tier & Onboarding:** `L1` (Lightweight Mode – Alapvédelem) | Onboarding: `A` (Admin-provisioned – `CORE-ONBOARD-
- **Indoklás (miért L1):** Saját használatú, 2 fős zárt belső mikro-alkalmazás. Nem kezel külső ügyfél-PII-t, különl
- **Regisztráció zárolása (`FIREBASE-AUTH-001`):** Onboarding Model `A` miatt a Firebase Console / Identity Platform
- **Érintett kollekciók és hozzáférési szabály (`isOwner` + `hasOnly`):**
  - Globális alapértelmezett tiltás (`match /{document=**} { allow read, write: if false; }` – `FIREBASE-RULES-001`)
  - `maintenanceLogs/{logId}`: közvetlen kliensoldali CRUD kizárólag hitelesített és megerősített e-mail című (`emai
- **Kliens query korlátok (`where` + `.limit(<=50)`):** Minden kliensoldali `list` lekérdezésben kötelező az explici
- **Titokkezelés és API-key korlátozás (HTTP Referrer + API scope):** Frontend kódban és gitben titok nem tárolható;
- **Költségvédelem és Mentés (Budget Alert + export):** Google Cloud Billing Budget Alert beállítása és rendszeres i
- **Kötelező negatív tesztek helye és eredménye (`assertFails`):** `tests/firestore.rules.spec.ts` (`REF-TEST-RULES-
- **Projektfrissesség (`package-lock.json` / SDK verziók ellenőrizve):** `UNKNOWN` – A referencia sablonok `firebase(
```

2. Szigorú `firestore.rules` és Kliens Lekérdezési Minta (`REF-RULES-BASE-001`)

Kötelező 7-elemű Metaadat-fejléc (`REF-MOD-001`)

1. **Template ID:** `REF-RULES-BASE-001` (adaptálva a `maintenanceLogs` kollekcióra)
2. **Alkalmazhatóság:** L1 belső karbantartási napló mikro-alkalmazás (2 fős, Onboarding Model A).
3. **Target SDK Baseline:** Firestore Security Rules `rules_version = '2'`, kliensoldali `firebase@^11.x` (Modular Web SDK). *Nem automatikus verziózár; egyeztetendő a célprojekt `package-lock.json` fájljával!*
4. **Biztonsági Invariánsok:**
   - Minden nem illeszkedő útvonal alapértelmezetten tiltott (`FIREBASE-RULES-001`).
   - A `maintenanceLogs` dokumentumok megkövetelik a tulajdonosi egyezést (`isOwner`), a `hasOnly` mező-engedélyező listát, a szigorú típus- és méretkorlátokat, a szerveroldali időbélyeget (`request.time`), valamint `update` során a `userId` és `createdAt` mezők megváltoztathatatlanságát (`immutability`).
   - A `list` lekérdezések megkövetelik a tulajdonosi/admin jogosultságot és a `request.query.limit <= 50` feltételt (`[FRAMEWORK DEFAULT] – FIREBASE-RULES-003`).
5. **Kötelező Tesztek:** Lásd a 3. pontban (`REF-TEST-RULES-001`).
6. **Ismert Korlátok:**
   - Az átfedő `allow` szabályok logikai `OR` kapcsolattal értékelődnek ki (`FIREBASE-RULES-001`).
   - A kliensoldali `.limit(50)` önmagában **nem biztonsági kontroll**, mert a kliens lekérdezés módosítható; a Firestore Rules a `request.query.limit <= 50` (`[FRAMEWORK DEFAULT] – FIREBASE-LIMIT-001`) feltétellel kényszeríti ki a hozzáférési korlátot, amely nem önálló költségvédelmi garancia (`FIREBASE-RULES-003`).
   - A Security Rules **nem korlátozza** a Firebase Admin SDK-t (`FIREBASE-ADMIN-001`).
7. **Verification Level & Last Verified:** `DOCUMENTATION_REVIEWED` | Utolsó dokumentációs felülvizsgálat: `2026-11-12` *(A célprojektben végrehajtott emulátoros futtatás szükséges az `EMULATOR_TESTED` szinthez!)*

A) `firestore.rules`

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Segédfüggvények (FIREBASE-AUTH-001, FIREBASE-RULES-001)
    function isAuthenticated() {
      return request.auth != null && request.auth.token.email_verified == true;
    }

    function isOwner(userId) {
      return isAuthenticated() && request.auth.uid == userId;
    }

    // Custom Claim admin ellenőrzés: nem igényel extra Firestore get() olvasást,
    // de a claim változása csak ID token frissítéskor lép életbe!
    // Custom Claims méretkorlát: max. 1 000 bájt [PLATFORM HARD LIMIT] (FIREBASE-LIMIT-001)
    function isAdmin() {
      return isAuthenticated() && request.auth.token.role == 'admin';
    }

    // 1. GLOBÁLIS FALLBACK DEFAULT DENY minden nem definiált útvonalra (FIREBASE-RULES-001)
    // Figyelem: az átfedő allow szabályok logikai OR szerint értékelődnek ki;
    // szülő útvonalra soha ne kerüljön túl megengedő szabály!
    match /{document=**} {
      allow read, write: if false;
    }

    // 2. FELHASZNÁLÓHOZ KÖTÖTT KARBANTARTÁSI NAPLÓ (L1: maintenanceLogs)
    match /maintenanceLogs/{logId} {
      allow get: if isOwner(resource.data.userId) || isAdmin();
      // request.query.limit <= 50: [FRAMEWORK DEFAULT] (FIREBASE-RULES-003, FIREBASE-LIMIT-001)
      allow list: if (isOwner(resource.data.userId) || isAdmin()) && request.query.limit <= 50;

      allow create: if isOwner(request.resource.data.userId)
        && request.resource.data.keys().hasOnly(['userId', 'title', 'notes', 'durationMinutes', 'createdAt', 'update
        && request.resource.data.title is string
        && request.resource.data.title.size() > 0
        && request.resource.data.title.size() <= 100
        && request.resource.data.notes is string
        && request.resource.data.notes.size() <= 2000
        && request.resource.data.durationMinutes is int
        && request.resource.data.durationMinutes > 0
        && request.resource.data.durationMinutes <= 600
        && request.resource.data.createdAt == request.time
        && request.resource.data.updatedAt == request.time;

      allow update: if isOwner(resource.data.userId)
        && request.resource.data.keys().hasOnly(['userId', 'title', 'notes', 'durationMinutes', 'createdAt', 'update
        && request.resource.data.userId == resource.data.userId // Tulajdonos (userId) módosíthatatlan (immutable)!
        && request.resource.data.createdAt == resource.data.createdAt // Létrehozás ideje (createdAt) módosíthatatla
        && request.resource.data.title is string
        && request.resource.data.title.size() > 0
        && request.resource.data.title.size() <= 100
        && request.resource.data.notes is string
        && request.resource.data.notes.size() <= 2000
        && request.resource.data.durationMinutes is int
        && request.resource.data.durationMinutes > 0
        && request.resource.data.durationMinutes <= 600
        && request.resource.data.updatedAt == request.time;

      allow delete: if isOwner(resource.data.userId) || isAdmin();
    }
  }
}
```

B) Kötelező Kliensoldali Lekérdezési Minta (`FIREBASE-RULES-003`)

Mivel a **Firestore Security Rules nem szűrő (Rules Are Not Filters)**, a kliensoldali lekérdezésnek explicit módon tartalmaznia kell mind a `where("userId", "==", currentUid)` szűrőt, mind a `.limit(<=50)` korlátot, különben a teljes lekérdezés `permission-denied` hibával elutasításra kerül:

```
import {
  Firestore,
  collection,
  query,
  where,
  orderBy,
```

```
  limit,
  getDocs,
} from "firebase/firestore";

// Kötelező kliensoldali lekérdezési minta (REF-RULES-BASE-001):
// Explicit userId szűrés + limit(<=50) megadása nélkül a Firestore Rules elutasítja a hívást!
export async function fetchMyMaintenanceLogs(db: Firestore, currentUid: string) {
  const q = query(
    collection(db, "maintenanceLogs"),
    where("userId", "==", currentUid),
    orderBy("createdAt", "desc"),
    limit(25) // <= 50 [FRAMEWORK DEFAULT] (FIREBASE-RULES-003)
  );
  return await getDocs(q);
}
```

3. Kötelező Pozitív és Negatív (`assertFails`) Emulátoros Tesztesetek (`REF-TEST-RULES-001`, `TEST-NEG-001`)

A teszteket kizárólag a **Firebase Local Emulator Suite** környezetben (`demo-*` projektazonosítóval) szabad futtatni, éles adatbázison soha (`TEST-FB-001`).

tests/firestore.rules.spec.ts

```
import {
  initializeTestEnvironment,
  assertSucceeds,
  assertFails,
} from "@firebase/rules-unit-testing";
import {
  doc,
  getDoc,
  setDoc,
  updateDoc,
  deleteDoc,
  collection,
  query,
  where,
  limit,
  getDocs,
  serverTimestamp,
} from "firebase/firestore";
import * as fs from "fs";

export async function runMaintenanceLogsRulesTests() {
  const testEnv = await initializeTestEnvironment({
    projectId: "demo-maintenance-security-test",
    firestore: {
      rules: fs.readFileSync("firestore.rules", "utf8"),
    },
  });

  // Kontextusok: hitelesített tulajdonos (Alice), másik felhasználó (Mallory),
  // nem megerősített e-mailű felhasználó, admin és névtelen (unauthenticated) kliens
  const aliceDb = testEnv
    .authenticatedContext("alice_uid", { email_verified: true })
    .firestore();
  const malloryDb = testEnv
    .authenticatedContext("mallory_uid", { email_verified: true })
    .firestore();
  const unverifiedDb = testEnv
    .authenticatedContext("alice_uid", { email_verified: false })
    .firestore();
  const adminDb = testEnv
    .authenticatedContext("admin_uid", { email_verified: true, role: "admin" })
    .firestore();
  const anonDb = testEnv.unauthenticatedContext().firestore();

  const initialCreatedAt = new Date("2026-11-12T10:00:00Z");

  // Kezdeti állapot betöltése a Security Rules megkerülésével (Admin SDK megfelelője)
  await testEnv.withSecurityRulesDisabled(async (context) => {
    const adminFs = context.firestore();
    await setDoc(doc(adminFs, "maintenanceLogs/log_1"), {
      userId: "alice_uid",
      title: "Klíma beltéri egység tisztítás",
      notes: "Szűrők mosása és fertőtlenítés",
      durationMinutes: 45,
      createdAt: initialCreatedAt,
      updatedAt: initialCreatedAt,
```

```
  });
});

// ================================================================================
// 1. POZITÍV TESZTEK (assertSucceeds)
// ================================================================================

// P1. Tulajdonos olvashatja a saját bejegyzését (get)
await assertSucceeds(getDoc(doc(aliceDb, "maintenanceLogs/log_1")));

// P2. Admin olvashatja a bejegyzést (get)
await assertSucceeds(getDoc(doc(adminDb, "maintenanceLogs/log_1")));

// P3. Tulajdonos szabályos sémával és szerveridővel létrehozhat új bejegyzést (create)
await assertSucceeds(
  setDoc(doc(aliceDb, "maintenanceLogs/log_valid_create"), {
    userId: "alice_uid",
    title: "Kompresszor ellenőrzés",
    notes: "Minden érték határértéken belül.",
    durationMinutes: 30,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// P4. Tulajdonos frissítheti a saját bejegyzését változatlan userId és createdAt mellett (update)
await assertSucceeds(
  updateDoc(doc(aliceDb, "maintenanceLogs/log_1"), {
    title: "Klíma beltéri egység teljes tisztítás",
    notes: "Szűrők mosása, hőcserélő fertőtlenítése",
    durationMinutes: 60,
    updatedAt: serverTimestamp(),
  })
);

// P5. Szabályos list lekérdezés tulajdonosi szűrővel és limit(20) <= 50 korláttal
const validQuery = query(
  collection(aliceDb, "maintenanceLogs"),
  where("userId", "==", "alice_uid"),
  limit(20)
);
await assertSucceeds(getDocs(validQuery));

// ================================================================================
// 2. KÖTELEZŐ NEGATÍV TESZTEK (assertFails – TEST-NEG-001)
// ================================================================================

// N1. Névtelen (nem hitelesített) olvasási és írási kísérlet -> MUST FAIL
await assertFails(getDoc(doc(anonDb, "maintenanceLogs/log_1")));
await assertFails(
  setDoc(doc(anonDb, "maintenanceLogs/log_anon"), {
    userId: "alice_uid",
    title: "Anonim bejegyzés",
    notes: "",
    durationMinutes: 15,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// N2. Nem megerősített e-mail című (email_verified: false) hozzáférés -> MUST FAIL
await assertFails(getDoc(doc(unverifiedDb, "maintenanceLogs/log_1")));

// N3. BOLA / IDOR: Másik felhasználó (Mallory) nem olvashatja, nem módosíthatja és nem törölheti Alice naplóját
await assertFails(getDoc(doc(malloryDb, "maintenanceLogs/log_1")));
await assertFails(
  updateDoc(doc(malloryDb, "maintenanceLogs/log_1"), {
    title: "Illetéktelen módosítás",
    updatedAt: serverTimestamp(),
  })
);
await assertFails(deleteDoc(doc(malloryDb, "maintenanceLogs/log_1")));

// N4. Spoofing: Mallory nem hozhat létre bejegyzést Alice userId-jával -> MUST FAIL
await assertFails(
  setDoc(doc(malloryDb, "maintenanceLogs/log_spoof"), {
    userId: "alice_uid",
    title: "Hamisított bejegyzés",
    notes: "",
```

```
    durationMinutes: 20,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// N5. Immutabilitás: Tulajdonos sem módosíthatja a userId mezőt update során -> MUST FAIL
await assertFails(
  updateDoc(doc(aliceDb, "maintenanceLogs/log_1"), {
    userId: "mallory_uid",
    updatedAt: serverTimestamp(),
  })
);

// N6. Immutabilitás: Tulajdonos sem módosíthatja a createdAt időbélyeget update során -> MUST FAIL
await assertFails(
  updateDoc(doc(aliceDb, "maintenanceLogs/log_1"), {
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// N7. Mass Assignment / hasOnly védelem: nem engedélyezett extra mező (isAdmin: true) beszúrása -> MUST FAIL
await assertFails(
  setDoc(doc(aliceDb, "maintenanceLogs/log_extra_field"), {
    userId: "alice_uid",
    title: "Jogosultság-kiterjesztési kísérlet",
    notes: "",
    durationMinutes: 30,
    isAdmin: true,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// N8. Érvénytelen értéktartomány (durationMinutes <= 0 vagy > 600, ill. üres title) -> MUST FAIL
await assertFails(
  setDoc(doc(aliceDb, "maintenanceLogs/log_invalid_range"), {
    userId: "alice_uid",
    title: "Hibás időtartam",
    notes: "",
    durationMinutes: -10,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);
await assertFails(
  setDoc(doc(aliceDb, "maintenanceLogs/log_empty_title"), {
    userId: "alice_uid",
    title: "",
    notes: "",
    durationMinutes: 30,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  })
);

// N9. Lekérdezés tulajdonosi szűrő (where('userId', '==', ...)) nélkül (unfilteredQuery) -> MUST FAIL
const unfilteredQuery = query(
  collection(aliceDb, "maintenanceLogs"),
  limit(20)
);
await assertFails(getDocs(unfilteredQuery));

// N10. Lekérdezés .limit() záradék nélkül (noLimitQuery) -> MUST FAIL
const noLimitQuery = query(
  collection(aliceDb, "maintenanceLogs"),
  where("userId", "==", "alice_uid")
);
await assertFails(getDocs(noLimitQuery));

// N11. Lekérdezés a megengedett maximumot (50) meghaladó limittel (limit(100) -> excessiveQuery) -> MUST FAIL
const excessiveQuery = query(
  collection(aliceDb, "maintenanceLogs"),
  where("userId", "==", "alice_uid"),
  limit(100)
);
await assertFails(getDocs(excessiveQuery));
```

```
  // N12. Default Deny ellenőrzése nem definiált kollekción -> MUST FAIL
  await assertFails(getDoc(doc(aliceDb, "otherCollection/doc_1")));

  await testEnv.cleanup();
}
```

Kötelező Biztonsági Összefoglaló Jelentés (`CORE-RESP-001`)

1. **Aktivált Modulok és Betöltött Sablonok (`CORE-LOAD-001`):**
   - **Tier-0 (Kötelező mag):** `01_SECURITY_CORE.md` (v1.1.0), `08_AI_AGENT_CODING_SECURITY.md` (v1.1.0)
   - **Tier-1 (Szelektív tartományi modulok):** `02_FIREBASE_SECURITY.md` (v1.1.0), `09_SECURITY_TESTING_THREAT_MODELING.md` (v1.1.0)
   - **Tier-2 (Implementációs és teszt réteg) & Sablonok:** `11_REFERENCE_TEMPLATES_FIREBASE.md` (`REF-RULES-BASE-001`, `REF-TEST-RULES-001`), `SECURITY_PROFILE_TEMPLATE_HU.md`
2. **Kockázati Szint, Onboarding Modell és Feltételezések:**
   - **Tier:** `L1` (Lightweight Mode – `CORE-TIER-L1-001`) | **Onboarding Modell:** `A` (Admin-provisioned – `CORE-ONBOARD-001`).
   - **Feltételezések:** 2 fős belső alkalmazás külső ügyfél-PII és pénzügyi/bérlet tranzakciók nélkül; a `maintenanceLogs` séma mezői: `userId`, `title`, `notes`, `durationMinutes`, `createdAt`, `updatedAt`.
3. **Érintett Fájlok és Zárolási Állapot (`CORE-CONC-001`):**
   - `allowed_files_to_modify`: `["SECURITY_PROFILE.md", "firestore.rules", "src/services/maintenanceLogs.ts", "tests/firestore.rules.spec.ts"]`
   - `lock_acquisition_mode`: `"COORDINATION_RESERVATION"` | `lock_released`: `true` (tervezési/referencia fázis lezárva).
4. **Érvényesített Biztonsági Kontrollok:**
   - `CORE-FRESH-001`: Háromszintű frissességi kapu ellenőrizve (`2026-11-12` < `2027-02-01`).
   - `CORE-TIER-L1-001`: Kompakt L1 profil a kötelező műszaki kontrollok (`isOwner`, `hasOnly`, immutabilitás, `assertFails`) megtartásával.
   - `FIREBASE-AUTH-001`: Onboarding Model `A` (*Disable User Sign-Up*) + `email_verified == true` + *Zero-Privilege Default Account*.
   - `FIREBASE-RULES-001` & `FIREBASE-RULES-003`: Globális alapértelmezett tiltás (`default deny`), `hasOnly` mezőlista, típus- és méretkorlátok, `userId`/`createdAt` immutabilitás, `request.query.limit <= 50` (`[FRAMEWORK DEFAULT]` – `FIREBASE-LIMIT-001`).
   - `TEST-FB-001` & `TEST-NEG-001`: Teljes pozitív és 12-pontos negatív (`assertFails`) emulátoros tesztfedettség.
5. **Verifikációs Bizonyítékok és Verifikációs Szint (`REF-VERIFY-001`, `CORE-REV-001`):**
   - `Verification Level`: **`DOCUMENTATION_REVIEWED`** (A referencia kódok és tesztek dokumentációs és logikai ellenőrzése megtörtént; tényleges lokális emulátoros tesztfuttatás ebben a munkamenetben nem zajlott).
   - `review_independence`: **`SELF_REVIEWED_L1_ONLY`** (L1 szinten megengedett).
6. **Nyitott Kérdések és Maradványkockázatok (`Residual Risks`):**
   - A célprojekt `package.json` és `package-lock.json` fájljai még nem lettek összevetve a `firebase@^11.x` és `@firebase/rules-unit-testing` verziókkal (`Project Freshness: UNKNOWN`).
   - Ha a karbantartási naplóban további mezőkre (pl. `deviceId`, `location` vagy `cost`) is szükség van, azokat a `firestore.rules` `hasOnly` listájában és a típus/méretkorlátok között is explicit módon fel kell venni.
   - Megosztott eszközök használata esetén a PWA kijelentkezéshez a `memoryLocalCache()` vagy a `REF-PWA-LOGOUT-001` (`BroadcastChannel` + `TAB_ID`) minta alkalmazása javasolt (`FIREBASE-PWA-001`).
7. **Kiadási Státusz (`Release Status` – `CORE-HANDOFF-001`):**
   - **`NOT_VERIFIED`** *(Kódgenerálási fázis: a `PASS` státusz kiadásához a `npx firebase emulators:exec 'npm test'` parancs tényleges, hibamentes lefuttatása szükséges a célprojektben).*

Szeretnéd, hogy kiegészítsük a `maintenanceLogs` sémáját konkrét mezőkkel (pl. gép/eszköz azonosító, helyszín), vagy elkészítsem hozzá a `PLANNER_TO_CODER` handoff szerződést a VS Code-os implementációhoz?
